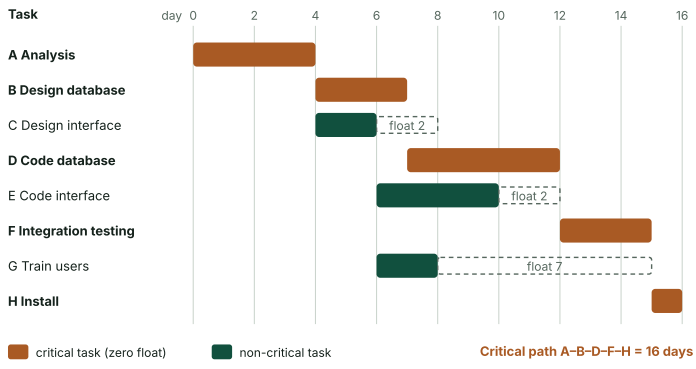
Organising and managing a software project

AP12 FACT FILE

Specification: students should be able to

- Describe the main elements in the organisation of a software project: resources, constraints and risks
- Explain the main features of project management for a software project, including the role of the project manager and SMART objectives
- Describe the roles of members of a software project team
- Construct and interpret Gantt charts and critical path analysis, identifying the critical path, earliest start times, latest finish times and float
- Evaluate the use of project management tools: Gantt charts and critical path analysis

Key words: resources, recurring resource, depleting resource, elastic resource, dedicated resource, constraints, scope, risk, requirements shift, staff turnover, project manager, SMART objectives, performance indicator, Gantt chart, critical path analysis (CPA), earliest start time (EST), latest finish time (LFT), float



Gantt chart for an eight-task project showing the critical path A, B, D, F, H of 16 days and the float on tasks C, E and G

Overview: resources, constraints and risks

Every software project, whichever methodology it follows, has the same central task: to achieve its objectives using the resources available, staying within its constraints and keeping risks under control. Resources are what the project can use: human resources such as programmers and database administrators, technical resources such as processing power and network bandwidth, financial resources such as the budget, and physical resources such as office space. Constraints are the limits placed on the project, usually concerning its scope, its deadline and the resources available. Risks are anything that could harm the project or cause it to fail, such as missing the deadline, shifting requirements, staff leaving or low productivity. The project manager is responsible for balancing all three. To do this they set clear, agreed objectives, plan and schedule the work, allocate staff and monitor progress. Tools such as Gantt charts and critical path analysis help them see how tasks depend on one another, which tasks can run in parallel and which tasks must not be delayed if the project is to finish on time.

Classifying and managing resources

As well as being human, technical, financial or physical, resources can be classified by how they can be used. By availability, a recurring resource such as a developer assigned to the project does not run down as it is used, whereas a depleting resource such as money or time is gradually consumed. By location, a fixed resource such as a server room cannot easily be moved, while a movable resource such as a member of staff can. By elasticity, an elastic resource such as the budget or the number of staff can in theory be increased or reduced, while an inelastic resource such as the time before a hard deadline cannot. By ownership, a dedicated resource works only on this one project, while a shared resource, such as a graphic designer, also works on others. Resource management has three activities. Planning estimates and schedules what will be needed. Deployment allocates resources to tasks and monitors progress, reallocating them if a stage slips. Release compares what was actually used with the plan, so that future estimates improve.

Constraints and risks

Constraints restrict what the developer can do. Scope constraints are characteristics fixed by the client, such as the number of simultaneous users, the response time required or the browsers and mobile platforms that must be supported. Time constraints set a deadline, so the project manager must check that the agreed scope can actually be delivered in the time available. Resource constraints limit budget, staff and equipment; even elastic resources have a ceiling in practice. Constraints must be stated explicitly, with an actual date and an actual maximum budget, because vague phrases such as ‘as quickly as possible’ are open to different interpretations. Risks must be identified and managed. Missing the deadline can cost the client business and, under a fixed-price contract, leave the developer paying for extra work. Requirements shift, often caused by users not being involved enough during analysis, leads to rework and extra cost. Staff turnover can remove vital knowledge, so collaboration and documentation should share it. Low productivity on long tasks is reduced by splitting them into smaller sections with their own deadlines.

Software project management and SMART objectives

Software project management emerged as a distinct role in the 1970s after a series of high-profile failures. It differs from managing a construction project in three ways: software is intangible, so progress is hard to see; most large systems are bespoke, so experience from one project transfers less well to the next; and technology changes so quickly that new tools may appear during the project. The project manager controls analysis, development and deployment, producing a realistic plan that sets out what will be built, the acceptance criteria and how the system will be maintained, then scheduling activities, allocating staff and managing risk. Success depends on clear objectives agreed by everyone at the start. Each objective needs a statement of what it should achieve, performance indicators that will measure it and the value of each indicator that counts as success. A single target value is better than a range, because a range lets each side claim a different result as success. The mnemonic SMART summarises this: objectives should be Specific, Measurable, Agreed, Realistic and Timed.

Gantt charts

Project management software helps the manager plan each task, delegate staff to tasks, monitor progress and analyse the schedule to spot problems before they happen. The Gantt chart is the most widely used of these tools. It is a horizontal bar chart in which each task is a bar placed against a timeline, so the position and length of the bar show the task’s start, duration and end. A column often records which tasks each task depends on. From a Gantt chart a manager can see what each task is, when it is scheduled to start and finish, how long it lasts, how far tasks overlap and which tasks depend on others. It is easy for clients and staff to understand, and it can show progress to date and the knock-on effect of a delay. In the worked example, the database coding can start only when the database design is finished, and integration testing must wait for both coding tasks. On a large chart, however, dependencies can be hard to follow and the critical path is not always obvious.

Critical path analysis

Critical path analysis models a project as a network. Nodes, drawn as circles, represent events such as the start or end of tasks, and arrows represent the tasks, labelled with their durations. Each node records its earliest start time and its latest finish time. In the forward pass, working from left to right, the earliest start time of each node is found from the tasks leading into it; where several arrows meet, the highest value is taken because every one of those tasks must be finished. The final node gives the project duration. In the backward pass, working from right to left, the latest finish time of the final node is set to the project duration and task durations are subtracted; where paths meet, the lowest value is taken. Nodes where the two times are equal are critical, and the tasks joining them form the critical path, the longest chain through the project. Any delay to a critical task delays the whole project. Non-critical tasks have float, the time they can slip without affecting the finish date.

T Key terms from the mark schemes

Resources
Anything a project can use to meet its objectives: human (staff), technical (e.g. processing power), financial (budget) or physical (e.g. accommodation).

Elastic resource
A resource whose amount can be increased or decreased, such as the budget or number of staff; an inelastic resource, such as time before a hard deadline, cannot.

Constraint
A restriction on a project that the developer cannot change, usually concerning scope, time or resources.

Risk
Anything that could have a harmful effect on a project or cause it to fail, such as missed deadlines, requirements shift, staff turnover or low productivity.

Requirements shift
Changes to what the client wants after the project has started, often caused by inadequate user involvement during analysis.
Credited: Scope creep

Project manager
The person responsible for planning, scheduling, resourcing, monitoring and managing risk so that a project is delivered on time, within budget and to the required quality.
Credited: Software project manager

SMART objectives
Objectives that are Specific, Measurable, Agreed, Realistic and Timed, so that all parties share the same view of success.

Gantt chart
A horizontal bar chart showing project tasks against a timeline, including their start and end, duration, overlaps, dependencies and milestones.

Critical path
The longest sequence of dependent tasks in a project, which determines the minimum time in which the project can be completed.

Earliest start time (EST)
In critical path analysis, the earliest time a task can start, once every task it depends on has finished.
Credited: EST

Latest finish time (LFT)
In critical path analysis, the latest time by which an activity must be complete if the project is not to be delayed.
Credited: LFT

Float (slack)
The amount of time a non-critical task can be delayed without delaying the completion of the whole project.
Credited: Slack

C C# example

Worked example: finding the critical path

```
string[] task = { "A", "B", "C", "D", "E", "F", "G", "H" };
int[] dur = { 3, 4, 2, 5, 6, 3, 2, 1 };
int[][] pre = { new int[0], new[] { 0 }, new[] { 0 }, new[] { 1 },
                new[] { 2 }, new[] { 3, 4 }, new[] { 2 }, new[] { 5, 6 } };
int[] est = new int[8];
for (int i = 0; i < 8; i++)          // forward pass
    foreach (int p in pre[i])
        est[i] = Math.Max(est[i], est[p] + dur[p]);
int end = est[7] + dur[7];            // 16 days
int[] lft = new int[8];
for (int i = 7; i >= 0; i--)          // backward pass
{
    lft[i] = end;
    for (int j = i + 1; j < 8; j++)
        if (Array.IndexOf(pre[j], i) >= 0)
            lft[i] = Math.Min(lft[i], lft[j] - dur[j]);
    Console.WriteLine($"{task[i]}: EST {est[i]}, float {lft[i] - dur[i] - est[i]}");
}
```

Exam focus

Expect to explain resources, constraints and risks with examples, expand SMART, draw a Gantt chart from a dependency table, complete a CPA diagram and identify the critical path, and explain the advantages of these tools. When explaining resources, constraints or risks, define the term and give a specific example; mark schemes credit the categories human, technical, financial and physical for resources, scope, time and resources for constraints, and time, requirements shift, staff turnover and productivity for risks. SMART should be expanded as Specific, Measurable, Agreed, Realistic and Timed, each with a brief explanation. On Gantt charts, check that no bar starts before every task it depends on has finished. On CPA diagrams, take the highest value when merging in the forward pass and the lowest in the backward pass, list the critical tasks in order and state the duration. When asked for advantages, link each feature to a benefit, for example identifying tasks that can run in parallel to minimise the project duration. A common error is giving the shortest path as the critical path; it is always the longest.

? Check yourself

1. State the four types of resource that may be available to a software project and give an example of each. [4]
2. Explain the difference between an elastic resource and an inelastic resource, using an example of each. [2]
3. Describe three constraints that may be placed on a software project. [3]

Answers: 1. Human, e.g. programmer / database administrator (1) / Technical, e.g. processing power / network bandwidth (1) / Financial, e.g. the budget (1) / Physical, e.g. office accommodation / transport links (1). Type and valid example needed for each mark. 2. Elastic: the amount can be increased or decreased, e.g. more money or staff can be assigned (1) / Inelastic: the amount cannot be changed, e.g. the time available before a fixed deadline (1). 3. Scope – characteristics of the product fixed by the client, e.g. number of simultaneous users, response time, browser compatibility (1) / Time – a fixed deadline by which the system must be delivered (1) / Resources – limits on budget, staff or equipment, e.g. a maximum budget (1).